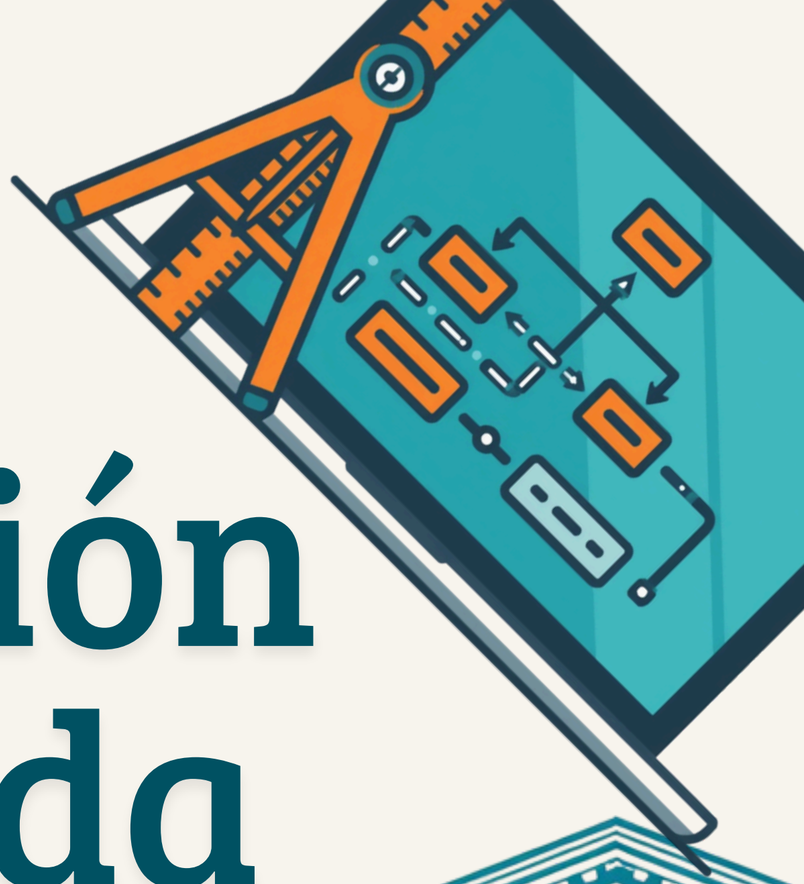
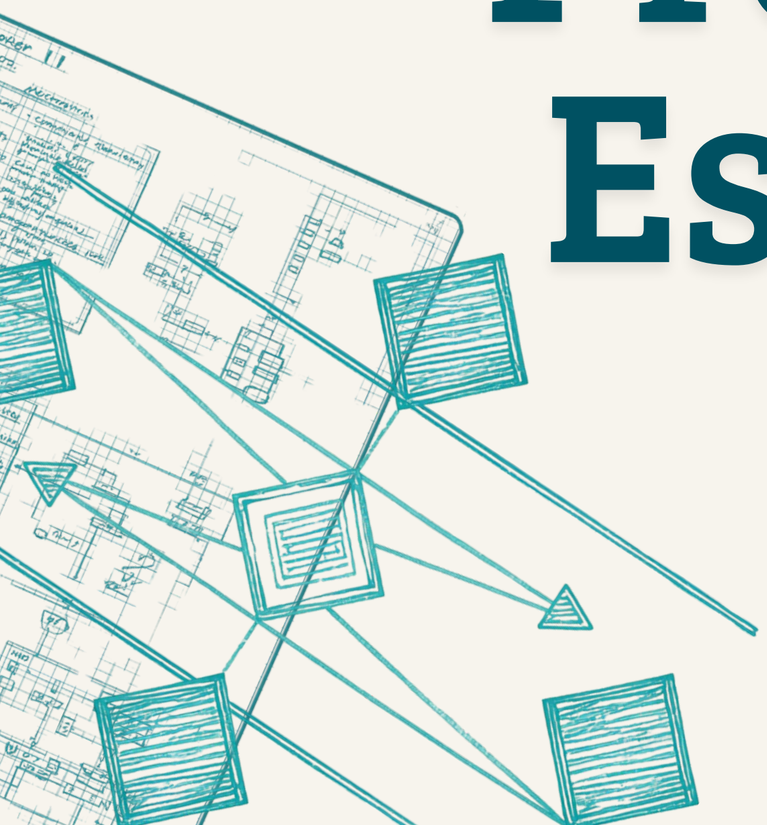


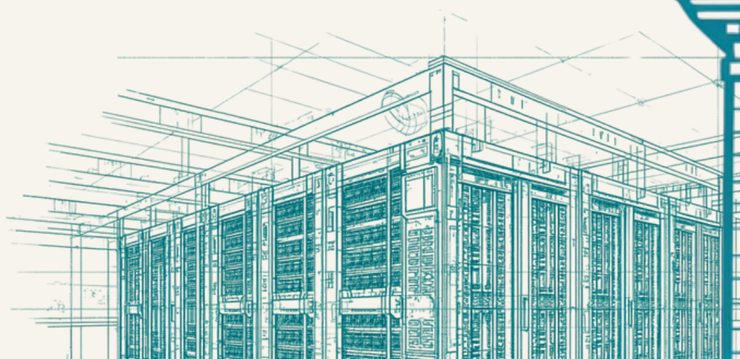




Programación Estructurada

M.C.C. Maritza Flores DOmínguez







Índice

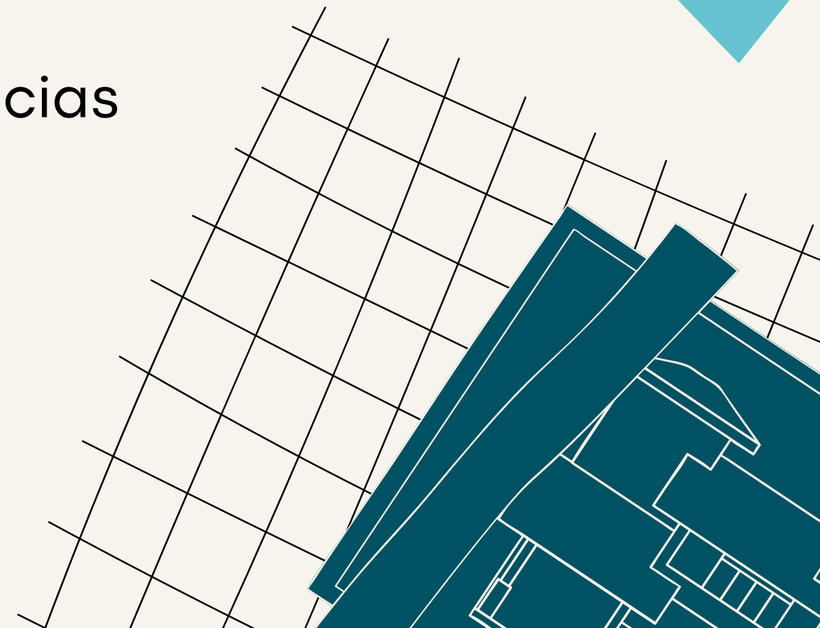
2.1.1 - 2.1.2 Identificadores y Tipos de Datos

2.1.3 - 2.1.4 Operadores, Operandos y Expresiones

2.1.5 Sentencias y Bloques de Sentencias







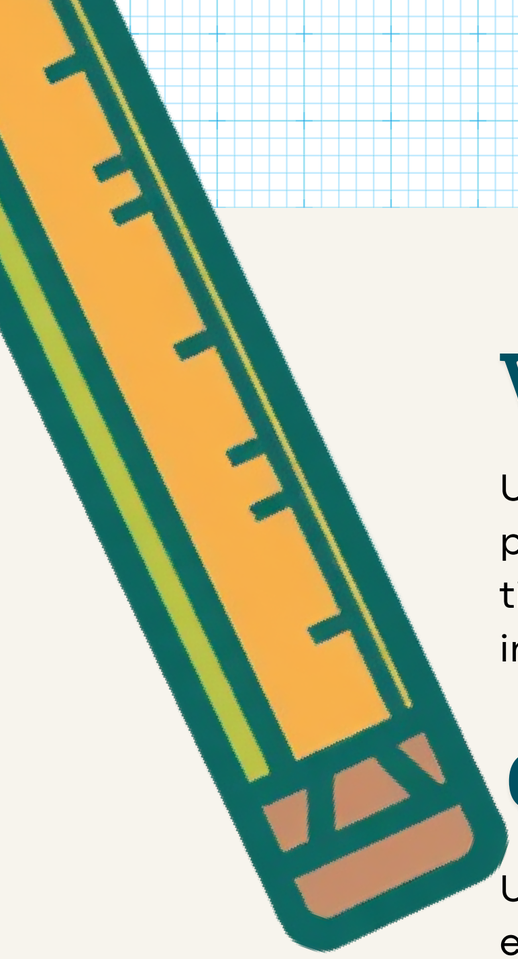
2.1.1 Identificadores

Variables

Una variable es un identificador que representa un espacio de memoria cuyo valor puede cambiar durante la ejecución del programa. Se declaran especificando su tipo y nombre (ej. int contador = 0;). Según Ceballos (2007), las variables deben inicializarse antes de usarse para evitar comportamiento indefinido en C/C++.

Constantes

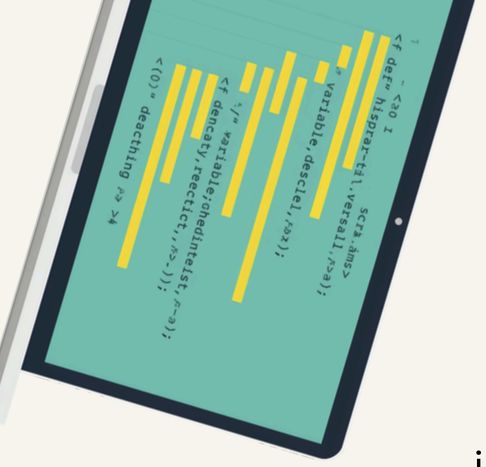
Una constante es un identificador cuyo valor permanece fijo durante toda la ejecución del programa. En C/C++ se definen con la directiva #define o con el calificador const (ej. const float PI = 3.1416;). Gottfried (2005) destaca que el uso de constantes mejora la legibilidad y el mantenimiento del código fuente.











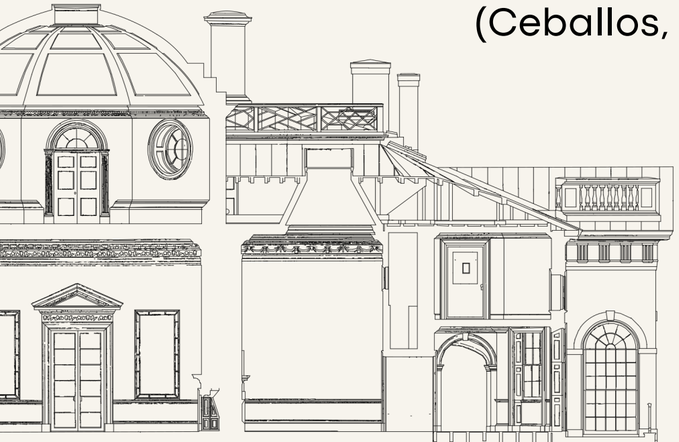
2.1.2 Tipos de Datos Simp

Enteros

int: Entero estándar (4 bytes). Rango: -2,147,483,648 a 2,147,483,647.
short: Entero corto (2 bytes). Rango: -32,768 a 32,767.
long: Entero largo (8 bytes). Mayor capacidad de almacenamiento.
unsigned: Variante sin signo, solo valores positivos.
Ejemplo: int edad = 25;
(Ceballos, 2007, p. 34)

Flotantes y Caracteres

float: Punto flotante simple (4 bytes). Precisión de 6-7 dígitos.
double: Doble precisión (8 bytes). Precisión de 15-16 dígitos.
long double: Precisión extendida (10-16 bytes).
char: Carácter (1 byte). Almacena un carácter ASCII. Rango: -128 a 127.
Ejemplo: double pi = 3.14159;
(Gottfried, 2005, p. 41)



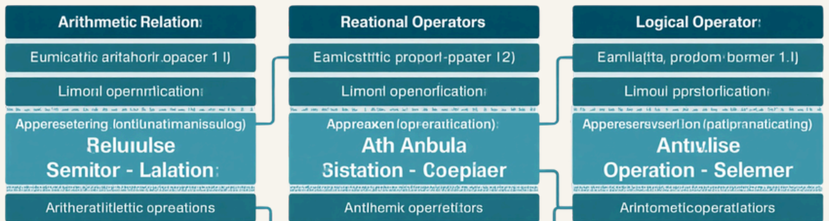




2.1.3 Operandos, Operadores y Precedencia

Operandos y Operadores:
Un operando es cualquier valor (variable, constante o expresión) sobre el que actúa un operador. Los operadores en C/C++ se clasifican en: Aritméticos (+, -, *, /, %), Relacionales (==, !=, <, >, <=, >=) y Lógicos (&&, ||, !). Ceballos (2007) señala que los operadores definen las operaciones a realizar entre operandos para producir un resultado.

Precedencia de Operadores:
El orden de evaluación determina cómo se agrupan los operandos. De mayor a menor precedencia: 1°) Paréntesis (), 2°) Unarios (!, -), 3°) Aritméticos (*, /, %), 4°) Aditivos (+, -), 5°) Relacionales (<, >, <=, >=), 6°) Igualdad (==, !=), 7°) Lógicos (&& luego ||). Gottfried (2005) recomienda usar paréntesis para claridad.
[5]







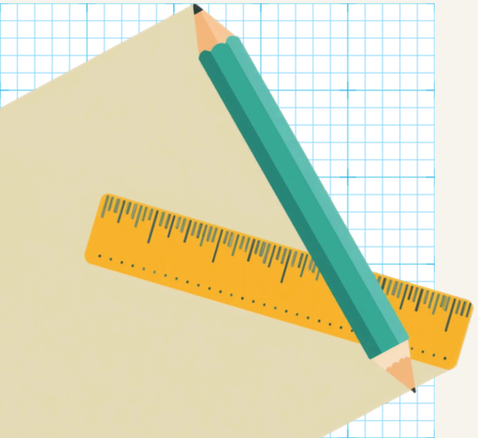


2.1.4 Construcc
Expresion

• Combinación de operandos y operadores: Una expresión se forma al combinar variables, constantes y operadores. Ejemplo: resultado = (a + b) * c - 5. La evaluación sigue las reglas de precedencia definidas por el lenguaje C/C++.



• Expresiones compuestas: Permiten realizar múltiples operaciones en una sola instrucción. Pueden incluir expresiones aritméticas, relacionales y lógicas combinadas. Ceballos (2007) señala que la correcta construcción garantiza resultados precisos y código legible.







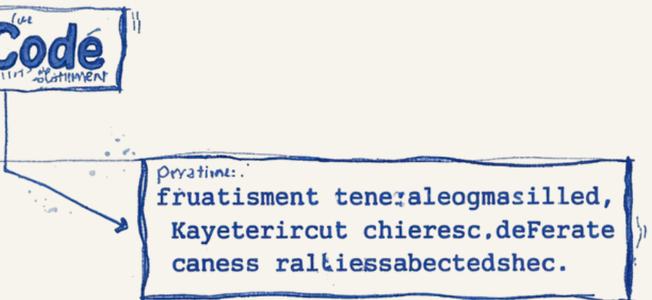
Sentencias y Bloques

2.1.5 Sentencias Simples

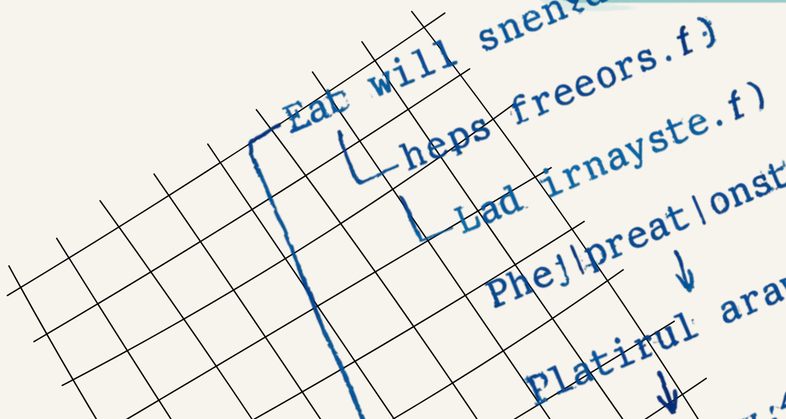
Una sentencia es la unidad básica de ejecución en C/C++. Las sentencias simples terminan obligatoriamente con punto y coma (;). Ejemplos: declaraciones de variables, asignaciones y llamadas a funciones. Ej: int x = 5; o printf("Hola"); Ceballos (2007) indica que cada instrucción representa una acción concreta del programa.

2.1.5 Bloques de Sentencias

Un bloque agrupa varias sentencias entre llaves { }. Define un ámbito (scope) propio para variables locales. Se usan en estructuras de control como if, for y while. Ej: { int a=1; int b=2; } Gottfried (2005) señala que los bloques permiten estructurar y modularizar el flujo de ejecución del programa.







Referencias

Ceballos, F. J. (2007). C/C++ Curso de Programación (3ra Ed.). Madrid: Ra-Ma Editorial. Esta obra aborda los fundamentos del lenguaje C/C++, incluyendo identificadores, tipos de datos, operadores, expresiones y estructuras de control propias de la programación estructurada.

• Gottfried, B. (2005). Programación en C (2da Ed.). Madrid: McGraw-Hill/Interamericana de España, S.A.U. Este texto cubre los conceptos esenciales de la programación en C: variables, constantes, tipos de datos simples, operandos, operadores, precedencia y construcción de expresiones y sentencias de programa.





¡Gracias por su atención!



